## 📘 Book 1 – Table of Contents

Chapter 01 Introduction to Git ..... Page 1

Chapter 02 Why Git? ..... Page 3

Chapter 03 Git vs GitHub ..... Page 5

Chapter 04 Version Control System ..... Page 8

Chapter 05 Installing Git ..... Page 12

Chapter 06 Git Configuration ..... Page 16

Chapter 07 Repository ..... Page 20

Chapter 08 git init ..... Page 24

Chapter 09 Git File States ..... Page 29

Chapter 10 git status ..... Page 35

Chapter 11 git add ..... Page 40

Chapter 12 Staging Area ..... Page 45

Chapter 13 git commit ..... Page 50

Chapter 14 Commit Messages ..... Page 56

Chapter 15 git log ..... Page 61

Chapter 16 git diff ..... Page 66

Chapter 17 git restore ..... Page 71

Chapter 18 git reset ..... Page 76

Chapter 19 .gitignore ..... Page 81

Chapter 20 GitHub ..... Page 87

Chapter 21 Creating a Repository ..... Page 92

Chapter 22 Local vs Remote Repository ..... Page 97

Chapter 23 git remote ..... Page 102

Chapter 24 git push ..... Page 107

Chapter 25 git pull ..... Page 112

Chapter 26 git clone ..... Page 117

Chapter 27 Branches ..... Page 122

Chapter 28 git branch ..... Page 127

Chapter 29 git checkout ..... Page 132

Chapter 30 git checkout -b ..... Page 137

## 📘 Handbook -Git & GitHub (Beginner Level)

Chapter 31 git merge ..... Page 142

Chapter 32 Merge Conflicts ..... Page 147

Chapter 33 Complete Workflow ..... Page 153

Chapter 34 Interview Questions ..... Page 159

Chapter 35 Command Reference ..... Page 166

---

---

# 📘 Git & GitHub Developer Cheat Sheet

## PART 1 – Git Fundamentals

---

# Chapter 1 – What is Git?

## 📌 Definition

**Git** is a **Distributed Version Control System (DVCS)** used to **track changes in files**, especially source code. It allows developers to save different versions of a project, collaborate with others, and restore previous versions if something goes wrong.

**Simple Meaning:** Git is like a **time machine** for your project.

---

## 🎯 Why was Git created?

Without Git:

- If you make a mistake, you may lose your old code.
- Working with multiple developers becomes difficult.
- It's hard to know **who changed what and when**.

Git solves all of these problems by keeping a complete history of your project.

---

# 🧠 Real-Life Analogy

Imagine you're writing a college assignment.

Without Git:

```
Assignment_Final.docx
Assignment_Final2.docx
Assignment_Final_Updated.docx
Assignment_Final_Last.docx
Assignment_Final_Last_Real.docx
```

Confusing, right?

With Git:

```
Version 1
   ↓
Version 2
   ↓
Version 3
   ↓
Latest Version
```

Every change is saved properly.

---

# 📊 Diagram

```
Project

↓

Edit Code

↓

Save Version (Commit)

↓

Edit Again

↓

Save New Version

↓

History Available Forever
```

---

# ⭐ Key Points

- Tracks file changes
- Saves project history
- Allows collaboration
- Helps recover deleted code
- Works locally on your computer

---

# 💻 Common Use Cases

- Software development
- Website development
- AI projects
- Python projects
- Team collaboration
- Portfolio projects

---

# ⚠ Common Mistake

❌ Git is **not** GitHub.

Many beginners think both are the same.

---

# 💼 Interview Question

**Q:** What is Git?

**Answer:**

Git is a Distributed Version Control System used to track changes in source code and manage project versions efficiently.

---

# 🧠 Shortcut

Remember:

**Git = Save + Track + Restore**

# Chapter 2 – Why Git?

## Problem Before Git

Developers used to manually create copies of projects.

Example:

```
Calculator

↓

Calculator_New

↓

Calculator_New_Final

↓

Calculator_Final_Final
```

After some time, nobody knows which version is correct.

---

## Git Solution

Git stores every version automatically.

```
Version 1

↓

Version 2

↓

Version 3

↓

Current Version
```

You can return to any version anytime.

---

## Benefits

✅ Complete history

✅ Easy collaboration

✅ Backup of project

✅ Undo mistakes

✅ Experiment safely

---

## Industry Use

Almost every software company uses Git, including projects hosted on GitHub, GitLab, or Bitbucket.

---

## ⭐ Remember

Git doesn't automatically save your work.

You must create a **Commit**.

---

## 💼 Interview Question

**Why is Git used?**

Git is used to track changes, collaborate with teams, maintain version history, and recover previous project states.

---

# Chapter 3 – Git vs GitHub

One of the most asked interview questions.

| Git | GitHub |
|---|---|
| **Software** | Website/Cloud Platform |
| **Installed on PC** | Runs on the Internet |
| **Tracks versions** | Stores repositories online |
| **Works offline** | Mostly requires internet |

| **Created by Linus Torvalds** | Owned by Microsoft |
|---|---|

---

# Easy Analogy

Git

↓

Your notebook

GitHub

↓

Cloud storage for your notebook

---

# Diagram

```
Laptop

Git Installed

↓

Repository

↓

git push

↓

GitHub Server
```

---

# Important Point

You can use Git without GitHub.

But you cannot use GitHub effectively without Git.

---

# Beginner Confusion

❌ Git saves projects online.

✓ Wrong.

Git saves projects locally.

GitHub stores them online.

---

## 💼 Interview Question

**Difference between Git and GitHub?**

Git is a version control software used locally.

GitHub is a cloud platform that hosts Git repositories.

---

# Chapter 4 – Version Control System (VCS)

## Definition

A Version Control System records every change made to files over time.

It allows developers to:

- Restore old versions
- Compare versions
- Track who changed files
- Collaborate safely

---

## Diagram

```
Version 1

↓

Version 2

↓

Version 3

↓

Version 4
```

Every version is stored.

---

## Types

### Local Version Control

Only one computer.

### Centralized Version Control

One central server.

### Distributed Version Control

Every developer has a complete copy.

Git is a **Distributed Version Control System (DVCS).**

---

## Why Distributed?

If the server crashes,

every developer still has the complete project history.

---

## ⭐ Remember

Git never stores only the latest version.

It stores the **entire history**.

---

# Chapter 5 – Git Workflow

This is the **most important concept** in Git.

Everything else revolves around this workflow.

---

# Workflow Diagram

```
Working Directory
        |
   git add
        ▼
Staging Area
        |
 git commit
        ▼
Local Repository
        |
  git push
        ▼
GitHub Repository
```

---

# Step-by-Step

## 1 Working Directory

Where you write or modify files.

Example:

```
main.py
index.html
style.css
```

---

## 2 Staging Area

Files are prepared for the next commit.

Command:

```
git add .
```

---

## 3 Local Repository

Git permanently saves the snapshot.

Command:

```
git commit -m "Added login feature"
```

---

### 🔼 Remote Repository (GitHub)

Upload local commits.

Command:

```
git push
```

---

# Complete Flow

```
Write Code

↓

git add

↓

git commit

↓

git push

↓

GitHub
```

---

# ⭐ Remember

**Nothing is saved in Git until you run `git commit`.**

- `git add` only stages changes.
- `git commit` creates a permanent snapshot.
- `git push` uploads commits to GitHub.

---

# 🔥 Most Important Commands (Introduced)

| Command | Purpose |
|---|---|
| `git add` | Stage changes |
| `git commit` | Save a snapshot |
| `git push` | Upload commits to GitHub |

---

## 📝 Mini Practice

1. Create a folder named `GitPractice`.
2. Create a file named `hello.py`.
3. Imagine the Git workflow:
    - Working Directory → `hello.py`
    - `git add hello.py`
    - `git commit -m "Add hello.py"`
    - `git push` (later, after connecting to GitHub)

---

# PART 2 – Git Setup & Your First Repository

---

# Chapter 6 – Installing Git

## 📌 Purpose

Git must be installed before you can track projects.

Without Git:

❌ No version control

❌ No commits

❌ No Git commands

---

## Installation Steps (Windows)

1. Download Git
2. Install with default settings

3. Finish installation
4. Open **Git Bash**

---

# Verify Installation

```
git --version
```

### Expected Output

```
git version 2.xx.x
```

✅ This means Git is installed correctly.

---

# 📊 Installation Flow

```
Download Git
        |
        ▼
Install
        |
        ▼
Open Git Bash
        |
        ▼
git --version
        |
        ▼
Ready to Use
```

---

# ⭐ Remember

Always verify Git after installation.

---

## ⚠ Common Mistake

❌ Installing Git but never checking:

```
git --version
```

---

## 💼 Interview Question

**How do you verify Git installation?**

```
git --version
```

---

# Chapter 7 – Git Bash

## 📌 What is Git Bash?

Git Bash is a terminal that allows you to run Git commands.

Instead of clicking buttons,
developers type commands.

---

## Why Git Bash?

Because Git is mainly command-line based.

Almost every tutorial and company uses Git commands.

---

## Diagram

```
Keyboard
      |
Type Command
      |
Git Bash
      |
Git Executes Command
```

---

# Example

```
pwd
```

Shows current folder.

---

```
ls
```

Shows files.

---

```
cd folder_name
```

Changes directory.

---

## ⭐ Most Useful Terminal Commands

| Command | Purpose |
|---|---|
| `pwd` | Current folder |
| `ls` | List files |
| `cd` | Change folder |
| `clear` | Clear screen |

---

## ⚠ Beginner Mistake

Opening **Command Prompt** instead of **Git Bash**.

Git Bash includes Git and Linux-style commands.

---

## 🧠 Shortcut

Think of Git Bash as:

**A remote control for Git.**

---

# Chapter 8 – Git Configuration (`git config`)

One-time setup after installing Git.

---

## Why Configure Git?

Every commit stores:

- Author Name
- Author Email

Git needs this information.

---

## Set Username

```
git config --global user.name "Khushi"
```

---

## Set Email

```
git config --global user.email "khushi@email.com"
```

---

## Verify Configuration

```
git config --list
```

Shows all Git settings.

---

## Check Only Username

```
git config user.name
```

---

## Check Only Email

```
git config user.email
```

---

## Diagram

```
Git
|
├── Username
└── Email
↓
Every Commit Stores These Details
```

---

## ⭐ Remember

Configure Git only once per computer.

---

## ⚠ Common Mistake

Wrong email address.

Later GitHub may not link your commits correctly.

---

## 💼 Interview Question

**Why is `git config` important?**

It tells Git who is creating commits.

---

# Chapter 9 – Repository

One of the most important Git concepts.

---

## Definition

A **Repository (Repo)** is a project that Git manages.

It stores:

- Files

- Folders
- Commit history
- Branches
- Project information

---

# Types

### Local Repository

Stored on your computer.

---

### Remote Repository

Stored on GitHub.

---

# Diagram

```
Local Repository
        |
git push
        ▼
Remote Repository
(GitHub)
```

---

# Real-Life Example

Repository = Your notebook.

Commits = Every saved version of that notebook.

---

# ⭐ Remember

One project = One Repository.

---

## 💼 Interview Question

**What is a Git Repository?**

A repository is a Git-managed project that stores files and complete version history.

---

# Chapter 10 – `git init`

This is where Git starts tracking a project.

---

## Definition

Creates a **new Git repository**.

---

## Syntax

```
git init
```

---

## What Happens?

Before:

```
Calculator Project
```

```
main.py
```

```
README.md
```

No Git.

---

Run:

```
git init
```

Now Git starts tracking.

---

# Diagram

```
Normal Folder
      |
git init
      ▼
Git Repository
```

---

# Output

Usually:

```
Initialized empty Git repository...
```

---

# What Gets Created?

A hidden folder:

```
.git
```

This is Git's brain.

Never delete it.

---

# ⚠ Common Mistake

Running:

```
git init
```

inside the wrong folder.

Always check current directory first.

```
pwd
```

---

# Mini Practice

Create folder

```
MyProject
```

Open Git Bash

Run

```
git init
```

Success.

---

## 💼 Interview Question

**What does `git init` do?**

Creates a new local Git repository.

---

# Chapter 11 – The `.git` Folder

Many beginners ignore this.

It is actually the most important folder.

---

## Definition

Hidden folder created after:

```
git init
```

---

## Stores

- Commit history
- Branches
- Configuration
- Objects
- References
- HEAD pointer

Everything Git needs.

---

## Diagram

```
MyProject
|
├── main.py
├── README.md
└── .git
      ├── commits
      ├── branches
      ├── config
      └── history
```

---

## ⚠ Never

Delete

`.git`

unless you want to remove Git completely.

---

## ⭐ Remember

No `.git`

=

No Git repository.

---

## 💼 Interview Question

**What is the purpose of the `.git` folder?**

It stores all repository metadata and version history used by Git.

---

# 🔥 Commands Covered in PART 2

| Command | Purpose |
|---|---|
| `git --version` | Check Git installation |

| | |
|---|---|
| `pwd` | Show current directory |
| `ls` | List files |
| `cd folder` | Change directory |
| `clear` | Clear terminal |
| `git config --global user.name` | Set username |
| `git config --global user.email` | Set email |
| `git config --list` | View Git configuration |
| `git init` | Create a new repository |

---

# 🧠 Memory Trick

```
Install Git

↓

Check Version

↓

Open Git Bash

↓

Configure Git

↓

Create Project Folder

↓

git init

↓

Git Repository Ready
```

---

# ⭐ Revision Box (30 Seconds)

- ✓ Install Git
- ✓ Verify with `git --version`
- ✓ Use Git Bash for commands
- ✓ Configure username and email once
- ✓ A repository is a Git-managed project
- ✓ `git init` creates a repository
- ✓ `.git` stores Git's entire history and metadata

- ✓ Never delete the `.git` folder unless you want to remove Git tracking

---

# 📘 Git & GitHub Developer Cheat Sheet

# PART 3 – Tracking Files & Your First Commit

💡 **Most Important Part of Git:**
Every developer uses these commands daily.

---

# Chapter 12 – File States in Git

## 📌 What are File States?

Every file in Git is always in **one of four states**.

Understanding these states is the **key to mastering Git**.

---

## 📊 Git File Lifecycle

```
Create File
     |
     ▼
Untracked
     |
 git add
     ▼
 Staged
     |
git commit
     ▼
 Tracked
     |
Edit File
     ▼
 Modified
     |
 git add
     ▼
 Staged
```

---

# 1️⃣ Untracked

## Meaning

Git knows the file exists, but **is not tracking it**.

Example:

```
login.py
```

You created it, but Git ignores it.

---

## How to Check

```
git status
```

Output:

```
Untracked files:
login.py
```

---

# 2️⃣ Staged

## Meaning

You told Git,

"Prepare this file for the next commit."

Command:

```
git add login.py
```

---

# 3️⃣ Tracked

## Meaning

After committing,

Git permanently tracks the file.

```
git commit -m "Added login.py"
```

---

# 4️⃣ Modified

## Meaning

You changed an already tracked file.

Example

Before

```
print("Hello")
```

After

```
print("Hello World")
```

Git detects the change.

---

# ⭐ Memory Trick

```
New File
↓
Untracked
↓
git add
↓
Staged
↓
git commit
↓
Tracked
↓
Edit
↓
Modified
```

---

## ⚠ Common Mistake

Many beginners think

```
git add
```

saves the project.

❌ Wrong

It only stages the file.

Only

```
git commit
```

creates a permanent version.

---

## 💼 Interview Question

**What are Git File States?**

Answer:

- Untracked
- Staged
- Tracked
- Modified

---

# Chapter 13 – `git status`

## 📌 Purpose

Shows the current status of your repository.

This is probably the **most frequently used Git command**.

---

## Syntax

```
git status
```

---

# What Does It Show?

- Current branch
- Untracked files
- Modified files
- Staged files
- Files ready for commit

---

# Example Output

```
On branch main

Untracked files:

login.py
```

---

After

```
git add login.py
```

Output becomes

```
Changes to be committed:
login.py
```

---

# Diagram

```
Repository
        |
git status
        |
        ▼
Shows Everything
```

---

# ⭐ When Should You Use It?

✓ Before adding

✓ Before committing

✓ Before pushing

✓ Whenever you're confused

---

## ⚠ Common Mistake

Developers forget to check

`git status`

before committing.

Always check first.

---

## 💼 Interview Question

**Why do we use `git status`?**

To view the current state of the repository and the status of files.

---

# Chapter 14 – `git add`

One of the most important commands.

---

## 📌 Purpose

Moves changes from the

Working Directory

↓

to

Staging Area.

---

# Workflow

```
Working Directory
        |
git add
        ▼
Staging Area
```

---

# Add One File

```
git add login.py
```

---

# Add Multiple Files

```
git add login.py
git add app.py
```

---

# Add Everything

```
git add .
```

This stages all changes in the current directory.

---

# Add All Python Files

```
git add *.py
```

---

# ⭐ Which One Should You Use?

| Command | Use When |
|---|---|
| `git add file.py` | One file |
| `git add .` | Entire project |
| `git add *.py` | Specific file type |

---

# ⚠ Common Mistake

Using

`git add .`

without checking changes.

Always run

`git status`

first.

---

## 💼 Interview Question

**What does `git add` do?**

Moves selected changes to the staging area for the next commit.

---

# Chapter 15 – Staging Area

This concept confuses many beginners.

---

## 📌 Definition

The Staging Area is a **temporary holding area**.

It lets you choose **what should be included in the next commit**.

---

## Why Does It Exist?

Imagine you changed

`login.py`

`style.css`

`README.md`

But today you only want to commit

`login.py`

The Staging Area allows this.

---

# Diagram

```
Working Directory

login.py

style.css

README.md

        |

git add login.py

        ▼

Staging Area

login.py
```

Only

```
login.py
```

will be committed.

---

# Real-Life Analogy

Shopping Cart

↓

Products on Shelf

↓

Add to Cart

↓

Checkout

The cart is like the Staging Area.

---

## ⭐ Remember

Git commits only

**Staged files**

---

## ⚠ Common Mistake

Thinking Git commits every modified file automatically.

It commits only staged files.

---

## 💼 Interview Question

**What is the Staging Area?**

A temporary area where files are prepared before creating a commit.

---

# Chapter 16 – First Commit

Now Git saves your work permanently.

---

## Syntax

```
git commit -m "First Commit"
```

---

## What Happens?

Git creates a snapshot.

Everything staged becomes part of project history.

---

## Workflow

```
Working Directory

↓

git add

↓

Staging Area

↓

git commit

↓

Repository History
```

---

# Good Commit Messages

```
git commit -m "Added login page"
git commit -m "Fixed navbar bug"
git commit -m "Updated README"
```

---

# Bad Commit Messages

```
git commit -m "Done"
git commit -m "abc"
git commit -m "Changes"
```

These don't explain anything.

---

# ⭐ Best Practice

Commit messages should explain

**what changed**

not

**how hard you worked.**

---

# ⚠ Common Mistake

Making one huge commit after 3 days.

Professional developers make

small,

meaningful commits.

---

## 💼 Interview Question

**What is a Commit?**

A commit is a permanent snapshot of staged changes stored in the repository.

---

# Chapter 17 – Commit Message (-m)

## 📌 Purpose

Adds a message describing the commit.

---

## Syntax

```
git commit -m "Added authentication"
```

---

## Why Important?

Months later,

you can understand

what happened.

---

## Good Examples

✅ Fixed login bug

✅ Added search feature

✅ Improved UI

---

## Bad Examples

❌ Work

❌ Update

❌ Done

---

## ⭐ Rule

Write commit messages in present/past action style.

Example:

Added

Fixed

Removed

Updated

Refactored

Created

---

# Chapter 18 – `git log`

## 📌 Purpose

Shows commit history.

---

## Syntax

```
git log
```

---

# Example Output

```
Commit ID

Author

Date

Commit Message
```

---

# Diagram

```
Newest Commit

↓

Older Commit

↓

Older Commit

↓

First Commit
```

---

# Useful Command

Compact view

```
git log --oneline
```

Example

```
a1234 Added Login

b5678 Fixed Bug

c9012 Initial Commit
```

---

# Why Useful?

- View history
- Find commit IDs
- Debug project
- Review progress

---

# 💼 Interview Question

**What is `git log` used for?**

Displays the complete history of commits in a Git repository.

---

# 🔥 Commands Covered in PART 3

| Command | Purpose |
|---|---|
| `git status` | Check repository status |
| `git add file` | Stage one file |
| `git add .` | Stage all changes |
| `git add *.py` | Stage all Python files |
| `git commit -m` | Create a commit |
| `git log` | Show commit history |
| `git log --oneline` | Compact commit history |

---

# 🧠 Complete Daily Git Workflow

```
Create File
      |
      ▼
git status
      |
      ▼
git add .
      |
      ▼
git status
      |
      ▼
git commit -m "Meaningful message"
      |
      ▼
git log
```

---

# ⭐ 30-Second Revision

- ✓ Every file has a Git state.
- ✓ `git status` tells you what's happening.
- ✓ `git add` moves files to the Staging Area.
- ✓ The Staging Area lets you choose what to commit.
- ✓ `git commit` permanently saves a snapshot.

- ✓ Use clear, meaningful commit messages.
- ✓ `git log` helps you view project history.

---

# 📘 Git & GitHub Developer Cheat Sheet

# PART 4 – Undoing Changes & Ignoring Files

💡 **Purpose:** Mistakes happen in every project. Git provides safe ways to undo changes without losing your work.

---

# Chapter 19 – `git diff`

## 📌 Purpose

Shows **what has changed** in your files before committing.

---

## Syntax

```
git diff
```

---

## What Does It Compare?

```
Last Commit
      |
      ▼
Current File
```

It highlights the differences between the two.

---

## Example

Before:

```
print("Hello")
```

After:

```
print("Hello World")
```

Running:

```
git diff
```

shows the exact lines that changed.

---

# Workflow

```
Modify File
      |
      ▼
git diff
      |
      ▼
Review Changes
```

---

# ⭐ Why Use It?

- Review code before committing
- Find accidental changes
- Check what exactly was modified

---

# ⚠ Common Mistake

Many beginners commit code **without reviewing changes**.

Always check:

```
git diff
```

---

# 💼 Interview Question

**What is `git diff` used for?**

It displays the differences between the current working files and the last saved version.

---

# Chapter 20 – `git restore`

## 📌 Purpose

Discard **uncommitted changes** and restore a file to its previous state.

---

## Syntax

```
git restore file.py
```

---

## Workflow

```
Modified File
      |
git restore
      ▼
Previous Version
```

---

## Example

Current file:

```
print("Wrong Code")
```

Run:

```
git restore app.py
```

Result:

```
print("Correct Code")
```

---

## ⭐ When to Use

✓ Wrong code written

✓ Accidental edits

✓ Want the last committed version

---

## ⚠ Warning

`git restore` removes **uncommitted changes permanently**.

There is no Undo after restoring.

---

## 🧠 Memory Trick

**Restore = Go back to the last committed version.**

---

## 💼 Interview Question

**What does `git restore` do?**

Restores a modified file to its last committed state by discarding uncommitted changes.

---

# Chapter 21 – `git reset`

## 📌 Purpose

Removes files from the **Staging Area**.

It does **not delete your code**.

---

## Syntax

```
git reset file.py
```

---

## Workflow

```
Working Directory
        |
git add
        ▼
Staging Area
        |
git reset
```

▼
Working Directory

---

# Example

You accidentally staged:

```
login.py
```

Undo staging:

```
git reset login.py
```

The file remains on your computer but is no longer staged.

---

# ⭐ Remember

```
git reset
```

❌ Doesn't delete your file.

✅ Only removes it from the staging area.

---

# ⚠ Common Mistake

Confusing

```
git restore
```

with

```
git reset
```

Difference:

| Command | Purpose |
|---|---|
| `git restore` | Undo file changes |
| `git reset` | Remove staged files |

---

# 💼 Interview Question

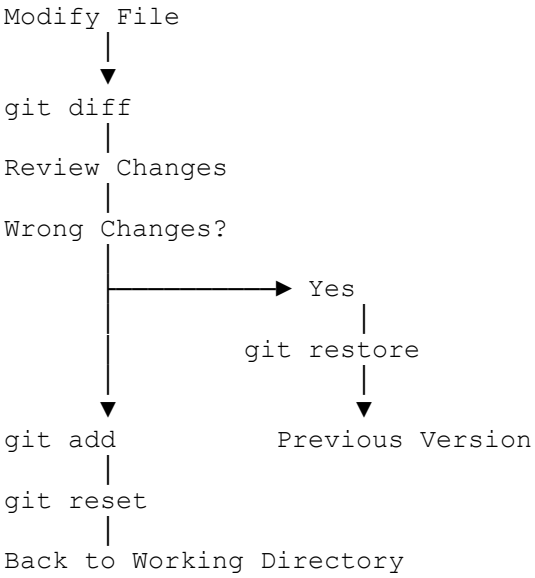
**What is `git reset` used for?**

It unstages files by moving them from the staging area back to the working directory.

---

# Chapter 22 – Undo Workflow

## 📊 Complete Flow




---

## ⭐ Quick Decision Guide

| Situation | Command |
|---|---|
| **Review changes** | `git diff` |
| **Undo file changes** | `git restore` |
| **Remove staged file** | `git reset` |

---

# Chapter 23 – `.gitignore`

One of the most useful files in Git.

---

## 📌 Purpose

Tells Git which files or folders should **not be tracked**.

---

# Why Needed?

Some files should never be uploaded.

Examples:

- Cache files
- Temporary files
- Build folders
- Personal configuration files

---

# Create

`.gitignore`

inside your project.

---

# Example

```
*.log

*.tmp

__pycache__/

.env
```

---

# Diagram

```
Project
|
├── main.py          ✅ Track
├── README.md        ✅ Track
├── .env             ❌ Ignore
├── debug.log        ❌ Ignore
└── __pycache__/     ❌ Ignore
```

---

## ⭐ Why Ignore Files?

- Keeps repository clean
- Prevents unnecessary uploads
- Protects sensitive information

---

## ⚠ Common Mistake

Adding secrets like:

`API Keys`

`Passwords`

`Tokens`

to GitHub.

Always ignore these using:

`.gitignore`

---

## 💼 Interview Question

**What is `.gitignore`?**

A file that tells Git which files or folders should not be tracked or committed.

---

# Chapter 24 – Best Practices for Undoing Changes

## ✅ Before Every Commit

```
git status
```

---

## Review Changes

```
git diff
```

---

## Stage Carefully

```
git add file.py
```

instead of always using:

```
git add .
```

---

## Undo Wrong Changes

```
git restore file.py
```

---

## Remove Wrongly Staged Files

```
git reset file.py
```

---

## Ignore Unnecessary Files

Create

```
.gitignore
```

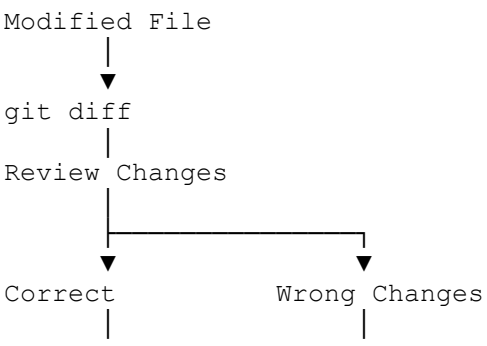
---

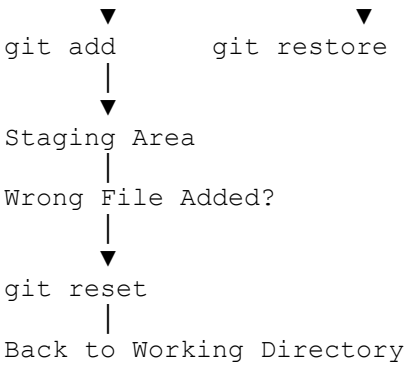
# 🔥 Commands Covered in PART 4

| Command | Purpose |
|---|---|
| **`git diff`** | Show file changes |
| **`git restore file`** | Undo file changes |
| **`git reset file`** | Remove from staging |
| **`.gitignore`** | Ignore files/folders |

---

# 📊 Complete Undo Cheat Sheet






---

# ⭐ 30-Second Revision

- ✓ `git diff` → See changes before committing.
- ✓ `git restore` → Discard uncommitted changes.
- ✓ `git reset` → Unstage files (does not delete them).
- ✓ `.gitignore` → Prevent Git from tracking unnecessary or sensitive files.
- ✓ Always review (`git diff`) and check status (`git status`) before committing.

---

# 💼 Interview Rapid Fire

### Q1. What is the difference between `git restore` and `git reset`?

**Answer:**

- `git restore` discards uncommitted changes in a file.
- `git reset` removes files from the staging area without deleting them.

---

### Q2. Why do we use `.gitignore`?

**Answer:**
To exclude files and folders that should not be tracked or committed.

---

### Q3. Which command helps you review changes before committing?

**Answer:**

`git diff`

---

# 📘 Git & GitHub Developer Cheat Sheet

# PART 5 – GitHub & Remote Repositories

💡 **Goal:** Until now, your code exists only on your computer (Local Repository). In this part, you'll learn how to store it online using **GitHub** and synchronize your local project with a remote repository.

---

# Chapter 25 – What is GitHub?

## 📌 Definition

**GitHub** is a cloud platform that hosts Git repositories online.

It allows developers to:

- Store code online
- Collaborate with others
- Share projects
- Backup repositories
- Contribute to open-source projects

---

## Git vs GitHub

| Git | GitHub |
|---|---|
| **Version Control Software** | Cloud Hosting Platform |
| **Installed on Computer** | Website |
| **Tracks Versions** | Stores Git Repositories |
| **Works Offline** | Mostly Online |

---

## 📊 Diagram

```
Your Computer
      |
      ▼
     Git
      |
 git push
      ▼
   GitHub
 (Cloud Storage)
```

---

## 🌍 Real-Life Analogy

Think of:

- **Git** = Microsoft Word (where you write)
- **GitHub** = Google Drive (where you upload and share)

---

## ⭐ Remember

Git tracks your project.

GitHub stores your project online.

---

## 💼 Interview Question

**What is GitHub?**

GitHub is a cloud platform used to host Git repositories for collaboration, backup, and version management.

---

# Chapter 26 – Creating Your First GitHub Repository

## 📌 Purpose

Before uploading code, you need a repository on GitHub.

---

## Steps

1️⃣ Login to GitHub

↓

2️⃣ Click **New Repository**

↓

3️⃣ Enter Repository Name

↓

4️⃣ Choose:

- Public
- Private

↓

5️⃣ Click **Create Repository**

---

# 📊 Flow

```
GitHub

↓

New Repository

↓

Repository Name

↓

Create

↓

Empty Remote Repository
```

---

# ⭐ Public vs Private

| Public | Private |
|---|---|
| **Anyone can view** | Only invited users |
| **Good for Portfolio** | Good for Company Projects |

---

# ⚠ Common Mistake

Don't initialize with README if you're connecting an existing local project (unless instructed). It can create unnecessary conflicts for beginners.

---

## 💼 Interview Question

**Why create a GitHub repository?**

To store and manage a project's Git history online.

---

# Chapter 27 – Local Repository vs Remote Repository

Many beginners confuse these.

---

## Local Repository

Located on your laptop.

```
Laptop

↓

Git Repository
```

Works even without internet.

---

## Remote Repository

Located on GitHub.

```
GitHub Server

↓

Remote Repository
```

Accessible from anywhere with permission.

---

## 📊 Complete Diagram

```
Local Repository
      |
git push
      ▼
Remote Repository
(GitHub)
      ▲
git pull
      |
Local Repository
```

---

## ⭐ Remember

| Local | Remote |
|---|---|
| **Computer** | GitHub |
| **Offline** | Online |
| **Fast** | Shared |

---

## 💼 Interview Question

**Difference between Local and Remote Repository?**

Local repositories exist on your computer.

Remote repositories are hosted online (GitHub).

---

# Chapter 28 – Connecting Local Repository to GitHub

Git doesn't automatically know where your GitHub repository is.

You must connect them.

---

## Command

```
git remote add origin <repository-url>
```

Example

```
git remote add origin https://github.com/username/project.git
```

---

# Breakdown

| Part | Meaning |
|---|---|
| **`git remote`** | Manage remote repositories |
| **`add`** | Add a new remote |
| **`origin`** | Default remote name |
| **URL** | GitHub repository link |

---

# Diagram

```
Local Repository
       |
git remote add origin
       ▼
Connected to
GitHub Repository
```

---

# Verify Connection

```
git remote -v
```

Expected Output

```
origin https://github.com/username/project.git
```

---

# ⭐ Remember

**origin** is simply the default nickname for your GitHub repository.

---

# ⚠ Common Mistake

Wrong repository URL.

Always copy the URL directly from GitHub.

---

## 💼 Interview Question

**What does `git remote add origin` do?**

It connects a local Git repository to a remote GitHub repository.

---

# Chapter 29 – `git push`

## 📌 Purpose

Uploads your local commits to GitHub.

---

## Syntax

```
git push origin main
```

---

## Workflow

```
Local Commit
      |
git push
      ▼
GitHub Repository
```

---

## What Gets Uploaded?

✓ Commits

✓ Files

✓ History

---

## ⭐ Important

Only committed changes are pushed.

```
Modified File

↓

git add

↓

git commit

↓

git push
```

---

## ⚠ Common Mistake

Trying:

```
git push
```

without making a commit first.

Nothing new will be uploaded.

---

## 💼 Interview Question

**What is `git push`?**

Uploads committed changes from the local repository to the remote repository.

---

# Chapter 30 – `git pull`

## 📌 Purpose

Downloads the latest changes from GitHub.

---

## Syntax

```
git pull origin main
```

---

# Workflow

```
GitHub
      |
git pull
      ▼
Local Repository
```

---

# Why Use It?

Before starting work:

Always get the latest code.

---

# ⭐ Best Practice

```
Open Project
↓
git pull
↓
Start Coding
```

---

# ⚠ Common Mistake

Developers edit old code without pulling latest updates.

---

# 💼 Interview Question

**What does `git pull` do?**

Downloads and updates your local repository with the latest changes from GitHub.

---

# Chapter 31 – `git clone`

## 📌 Purpose

Copies an existing GitHub repository to your computer.

---

## Syntax

```
git clone https://github.com/username/project.git
```

---

## Workflow

```
GitHub
      |
git clone
      ▼
Local Computer
```

---

## What Happens?

Git automatically:

✓ Downloads files

✓ Creates repository

✓ Connects remote

---

## ⭐ Use Cases

- Download open-source projects
- Join team projects
- Continue work on another computer

---

# ⚠ Common Mistake

Using:

```
git init
```

after cloning.

❌ Don't do this.

Clone already contains a Git repository.

---

# 💼 Interview Question

**What does `git clone` do?**

Creates a complete copy of a remote Git repository on your local machine.

---

# 📊 Complete Git → GitHub Workflow

```
Create Project
      |
git init
      |
git add
      |
git commit
      |
Create GitHub Repository
      |
git remote add origin
      |
git push
      ▼
```

GitHub

---

# 🔥 Commands Covered in PART 5

| Command | Purpose |
|---|---|
| `git remote add origin` | Connect local repo to GitHub |
| `git remote -v` | View remote repositories |
| `git push origin main` | Upload commits |
| `git pull origin main` | Download latest changes |
| `git clone <url>` | Copy repository |

---

# 💼 Interview Rapid Fire

### Q1. Difference between `git clone` and `git init`?

| git init | git clone |
|---|---|
| Creates new repository | Copies existing repository |
| Empty history | Complete history |
| Manual setup | Ready to use |

---

### Q2. What is `origin`?

The default name of the remote GitHub repository.

---

### Q3. What is the difference between `git push` and `git pull`?

| Command | Direction |
|---|---|
| `git push` | Local → GitHub |
| `git pull` | GitHub → Local |

---

# ⭐ 30-Second Revision

- ✓ GitHub stores repositories online.
- ✓ Create a repository before pushing code.
- ✓ `git remote add origin` connects local Git to GitHub.
- ✓ `git push` uploads commits.
- ✓ `git pull` downloads the latest changes.

- ✓ `git clone` copies an existing repository.

---

Absolutely! From this point onward, I'm keeping the cheat sheet **strictly based on the transcript**. I won't introduce advanced Git topics unless they're covered in your source.

---

# 📘 Git & GitHub Developer Cheat Sheet

# PART 6 – Branching & Collaboration

💡 **Goal:** Branches allow developers to work on new features or bug fixes without affecting the main project.

---

# Chapter 32 – What is a Branch?

## 📌 Definition

A **Branch** is an independent line of development inside a Git repository.

It lets you work on new code without changing the main project.

---

## Why Use Branches?

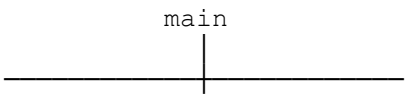
Instead of editing the main project directly, developers create a branch.

Benefits:

- Develop new features safely
- Fix bugs independently
- Test ideas without affecting stable code

---

## 📊 Diagram



```
          |
feature-login
```

Changes made in `feature-login` do not affect `main` until merged.

---

## 🌎 Real-Life Analogy

Imagine you're writing a book.

Instead of editing the original manuscript directly:

```
Original Book
      |
      ├── New Ending
      ├── Grammar Fix
      └── New Chapter
```

Each copy is like a branch.

---

## ⭐ Remember

One feature = One branch

---

## 💼 Interview Question

**What is a Git Branch?**

A branch is a separate line of development that allows changes without affecting the main project.

---

# Chapter 33 – View Existing Branches

## Command

```
git branch
```

---

## Purpose

Shows all branches in the repository.

---

## Example Output

```
* main
```

The * indicates the current branch.

---

## Diagram

```
Repository
      |
git branch
      ▼
* main
```

---

## ⭐ Remember

* = Current active branch.

---

## 💼 Interview Question

**How do you list all branches?**

```
git branch
```

---

# Chapter 34 – Create a New Branch

## Syntax

```
git branch feature-login
```

---

## What Happens?

Git creates a new branch.

It **does not switch** to it.

---

# Diagram

```
Before

main

After

main

feature-login
```

---

# Verify

```
git branch
```

Output

```
* main

feature-login
```

You are still on `main`.

---

# ⭐ Remember

```
git branch
```

creates only.

It doesn't switch.

---

# ⚠ Common Mistake

Many beginners think:

```
git branch feature-login
```

automatically changes the branch.

❌ It doesn't.

---

## 💼 Interview Question

**Does `git branch` switch branches?**

No.

It only creates a new branch.

---

# Chapter 35 – Switch Branch

## Syntax

```
git checkout feature-login
```

---

## Purpose

Moves from one branch to another.

---

## Diagram

```
main
      |
git checkout feature-login
      ▼
feature-login
```

---

## Verify

```
git branch
```

Output

```
main
```

```
* feature-login
```

---

## ⭐ Remember

`checkout`

=

Switch Branch

---

## ⚠ Common Mistake

Trying to checkout a branch that doesn't exist.

---

## 💼 Interview Question

**How do you switch branches?**

```
git checkout branch-name
```

---

# Chapter 36 – Create & Switch Together

Instead of two commands,

Git allows one shortcut.

---

## Syntax

```
git checkout -b feature-login
```

---

## What Happens?

This command:

✅ Creates branch

AND

✅ Switches to it.

---

## Diagram

```
main
      |
git checkout -b feature-login
      ▼
feature-login
```

---

## ⭐ Shortcut

Instead of

```
git branch feature-login
git checkout feature-login
```

Use

```
git checkout -b feature-login
```

---

## 💼 Interview Question

**What does `git checkout -b` do?**

Creates a new branch and immediately switches to it.

---

# Chapter 37 – Merge Branches

## 📌 Purpose

Combines changes from one branch into another.

---

## Example

```
main
↓
feature-login
↓
Completed
```

Now merge it.

---

# Syntax

```
git merge feature-login
```

---

# Diagram

```
main
|
├———feature-login
|
Merge
|
▼
main
```

---

# Workflow

1. Complete feature.
2. Switch to `main`.
3. Merge the feature branch.

---

# ⭐ Remember

Merge brings branch changes into the current branch.

---

## ⚠ Common Mistake

Merging while on the wrong branch.

Always check:

```
git branch
```

before merging.

---

## 💼 Interview Question

**What is `git merge`?**

It combines the history and changes of one branch into another.

---

# Chapter 38 – Merge Conflict

## 📌 What is a Merge Conflict?

A conflict happens when two branches modify the same part of a file differently.

Git cannot decide which version to keep.

---

## Example

Main branch

```
print("Hello")
```

Feature branch

```
print("Hello World")
```

Git needs you to choose the correct version.

---

## Diagram

```
main
|
Hello
|
feature
|
Hello World
|
Merge
|
Conflict
```

---

# How to Resolve

1. Open the conflicted file.
2. Choose the correct code.
3. Save the file.
4. Add the file.
5. Commit the merge.

---

# ⭐ Remember

Git detects the conflict.

You resolve it.

---

# 💼 Interview Question

**What is a Merge Conflict?**

A merge conflict occurs when Git cannot automatically combine changes from different branches.

---

# 📊 Complete Branch Workflow

```
main

|

Create Branch

|

▼

feature-login

|

Write Code

|

Commit

|

Switch to main

|

Merge

|

Updated main
```

---

# 🔥 Commands Covered in PART 6

| Command | Purpose |
|---|---|
| **`git branch`** | View branches |
| **`git branch name`** | Create branch |
| **`git checkout name`** | Switch branch |
| **`git checkout -b name`** | Create & switch |
| **`git merge name`** | Merge branch |

---

# 🧠 Branching Cheat Sheet

```
View Branches

↓

git branch
```

```
↓
Create Branch
↓
git branch feature
↓
Switch
↓
git checkout feature
↓
Commit Changes
↓
Checkout main
↓
git merge feature
```

---

# ⭐ 30-Second Revision

- ✓ Branches isolate work from the main project.
- ✓ `git branch` lists or creates branches.
- ✓ `git checkout` switches branches.
- ✓ `git checkout -b` creates and switches in one step.
- ✓ `git merge` combines branch changes.
- ✓ Merge conflicts happen when the same code is changed differently in two branches.

---

# 💼 Interview Rapid Fire

### Q1. Why do developers use branches?

To develop features or fix bugs without affecting the main codebase.

---

### Q2. Difference between `git branch` and `git checkout`?

| Command | Purpose |
|---|---|
| `git branch` | Create/List branches |
| `git checkout` | Switch branches |

---

### Q3. What is a merge conflict?

A situation where Git cannot automatically merge changes because the same part of a file was modified differently.

---

# ✅ End of PART 6

At this point, your transcript has covered the complete beginner workflow:

```
Install Git
      |
      ▼
Configure Git
      |
      ▼
git init
      |
      ▼
git status
      |
      ▼
git add
      |
      ▼
git commit
      |
      ▼
GitHub Repository
      |
      ▼
git push / git pull
      |
      ▼
Branch
      |
      ▼
Merge
```

This completes the core Git learning path reflected in your transcript. If later sections of your transcript introduce additional topics (such as GitHub collaboration, pull requests, SSH, or other workflows), we'll continue in the same concise, interview-focused cheat sheet format based only on those sections.